

Mandos Display Design Specification

Version: 2.0
Last updated: 2026-03-28
Purpose: Single source of truth for implementing `.show()`, `.compare()`, and `.report()` output rendering.
Audience: Developer copilots, contributors, reviewers.

1. Overview

Mandos has three display methods. All share one component library but differ in context and theme.

Method	Context	Theme	Output Format	Purpose
<code>.show()</code>	Jupyter notebook	Dark	Inline HTML widget	ydata-profiling style column stats, DQ flags, correlations
<code>.compare()</code>	Jupyter notebook	Dark	Inline HTML widget	Drift analysis comparing reference vs current datasets
<code>.report()</code>	Standalone file	Light	PDF (via HTML→PDF)	Audit-grade report for MRM validators and regulators

Design Principles

- Verdict-first.** Every view opens with a status verdict and summary counts. Never bury the conclusion.
- Audit-traceable.** Every metric displays its detection method, threshold, and observed value. Run metadata captures full data lineage.
- Shared components.** Badge, Card, Table, Verdict, CSIBar, DistChart, Legend, RunMeta are identical across all three views — only the `dark` flag changes.
- Scannable.** Tables sort by severity (critical first). Charts are inline in table rows, not on separate pages. Section numbering is consistent for cross-referencing.

5. **Accessible.** Status indicators use color + dot/icon + text label (never color alone).
Meets colorblind accessibility.
-

2. Color Theme

2.1 Status Colors (shared across all views)

Status	Hex	Use
Critical	#EF4444	Threshold breaches, high-severity drift
Warn	#F59E0B	Moderate issues, approaching thresholds
OK	#10B981	Passing checks, healthy metrics
Info	#6366F1	Neutral annotations, section accents

Each status color has derived variants:

```
subtle:  rgba({color}, 0.07)  - background fill for badges, banners
border:  rgba({color}, 0.22)  - border color for badges, banners
glow:    rgba({color}, 0.12)  - background fill for dark-theme cards
```

2.2 Dark Theme (`.show()` and `.compare()`)

Element	Value
Page background	#0C1018
Card/panel background	<code>rgba(255,255,255, 0.025)</code>
Card border	<code>rgba(255,255,255, 0.06)</code>
Table header background	<code>rgba(255,255,255, 0.03)</code>
Table row border	<code>rgba(255,255,255, 0.03)</code>
Primary text	#E6EDF3
Secondary text	<code>rgba(255,255,255, 0.35)</code>

Muted text	<code>rgba(255,255,255, 0.25)</code>
Section divider	<code>linear-gradient(90deg, rgba(99,102,241,0.3), transparent)</code>

2.3 Light Theme (`.report()`)

Element	Value
Page background	<code>#FFFFFF</code>
Card/panel background	<code>#FFFFFF</code>
Card border	<code>#E5E7EB</code>
Table header background	<code>#F9FAFB</code>
Table row border	<code>#F3F4F6</code>
Primary text	<code>#111827</code>
Secondary text	<code>#6B7280</code>
Muted text	<code>#9CA3AF</code>
Section divider	<code>linear-gradient(90deg, rgba(99,102,241,0.15), transparent)</code>

2.4 Comparison Colors

Role	Hex	Use
Reference (baseline)	<code>#6366F1</code> (indigo)	Left bars in overlays, baseline distributions
Current (production)	<code>#F97316</code> (orange)	Right bars in overlays, current distributions

2.5 Correlation Colors

Direction	Color
Positive correlation	<code>#A78BFA</code> (violet)

Negative correlation

#FBBF24 (amber)

3. Typography

3.1 Font Families

Role	Font	Fallback	Use
UI / prose	IBM Plex Sans	-apple-system, sans-serif	Labels, headings, descriptions, body text
Data / code	JetBrains Mono	Fira Code, monospace	Feature names, metric values, thresholds, code, run metadata

3.2 Font Sizes

Element	Size	Weight
Mandos logo text	18px	800
Section heading	16px	800
Card value	24px	800
Table body	11-12px	400-600
Badge text	10px	700
Card label	10px	600
Muted annotations	10px	400
Run metadata	11px	500

3.3 Special Settings

- **Tabular numerals:** All numeric values use `font-feature-settings: "tnum"` for column alignment.
- **Letter spacing:** Labels and badges use `letter-spacing: 0.05-0.06em`. Logo uses `-0.03em`.

- **Text transform:** Labels, badge text, and column headers are `text-transform: uppercase`.
-

4. Shared Components

4.1 Status Badge

A small pill with a colored dot + uppercase status label.

Layout: `[●] STATUS`
Shape: `border-radius: 4px`
Padding: `1px 8px (sm) or 3px 12px (lg)`
Dot: `5px circle, filled with status color`
Font: `sans, 10px, weight 700, uppercase, 0.05em spacing`
Colors: `background = status.subtle, text = status.bg, border = status.border`

4.2 Metric Card

A summary stat box used in the overview row.

Layout: `Vertical stack: LABEL → VALUE → sub-text`
Shape: `border-radius: 8px`
Padding: `14px 18px`
Flex: `flex: 1 1 130px (wraps on small screens)`
Accent: `Optional 3px left border in status color`
Label: `sans, 10px, weight 600, uppercase, 0.06em spacing, secondary color`
Value: `sans, 24px, weight 800, tnum, primary or status color`
Sub-text: `sans, 10px, muted color`

Cards appear in a horizontal flex row (gap: 8px) immediately below the verdict.

4.3 Table

A bordered table with header row and alternating subtle borders.

Container: `border-radius: 8px, overflow: hidden, 1px border`
Header row: `Slightly tinted background, uppercase labels`
`Font: sans, 10px, weight 700, uppercase, 0.06em spacing`
`Color: secondary text`
Body cells: `Font: sans, 11-12px (compact: 11px)`
`Padding: 9px 14px (normal) or 6px 10px (compact)`
Row borders: `1px solid, very subtle`

Compact mode (`compact=true`) reduces padding and font size for dense data.

4.4 Verdict Banner

A prominent alert banner at the top of each view.

Layout:	Horizontal: [ICON] [Text block]
Shape:	border-radius: 8px
Padding:	14px 20px
Icon:	36x36px rounded square (radius: 8px), filled with status color Glyph: Δ (critical), Δ (warn), $\checkmark$ (ok)
Title:	sans, 13px, weight 800 Text: "ACTION REQUIRED" "REVIEW RECOMMENDED" "HEALTHY"
Subtitle:	sans, 11px, secondary color Text: summary counts (e.g. "18 feature(s) drifted · 35 DQ issues")
Background:	status.glow (dark) or status.subtle (light)
Border:	1px solid status.border

4.5 Section Header

Numbered section headings with a gradient divider.

Layout:	[N.] Title Optional subtitle below, indented
Number:	mono, 12px, weight 800, color: #6366F1
Title:	sans, 16px, weight 800, -0.02em spacing
Subtitle:	sans, 11px, secondary color, margin-left: 22px
Divider:	1px height, gradient from indigo to transparent
Spacing:	margin-top: 28px, margin-bottom: 14px

4.6 CSI Bar (Horizontal)

A horizontal bar showing CSI magnitude for a single feature.

Layout:	[Feature name, right-aligned] [bar] [value]
Name:	mono, 11px, width: 200px, right-aligned
Bar:	height: 16px, border-radius: 3px Background: subtle gray Fill: gradient based on severity >10: red gradient (#EF4444) 1-10: amber gradient (#F59E0B) <1: indigo gradient (#6366F1) Width: proportional to maxCSI (default 25)
Value:	mono, 11px, weight 700, right-aligned, width: 48px Color matches the fill gradient

4.7 Distribution Chart (single dataset)

A simple bar chart for column profiles.

Type: Vertical bar chart (SVG)
Bars: Rounded corners (rx: 1), 1px gap between bars
Color: rgba(99,162,241, 0.6) default
Sizing: Configurable height (default 40px) and width (default 160px)
Data: Array of numeric values representing bin counts

4.8 Distribution Chart (dual overlay)

Side-by-side bars for reference vs current comparison.

Type: Grouped vertical bar chart (SVG)
Layout: Each bin has two bars side by side
Left bar (45% of bin width): Reference, color #6366F1, opacity 0.6
Right bar (45% of bin width): Current, color #F97316, opacity 0.7
Sizing: Configurable, default h=50, w=200

This chart appears inline in table cells for compact comparison.

4.9 Correlation Bar

A diverging horizontal bar showing Spearman correlation.

Layout: [Feature name] [diverging bar] [value]
Name: mono, 11px, width: 220px
Bar: width: 160px, height: 7px, border-radius: 3px
Center line at 50% (1px, subtle)
Positive fills right from center in violet gradient
Negative fills left from center in amber gradient
Value: mono, 11px, weight 700, width: 48px
Color: violet (positive) or amber (negative)
Format: signed with 3 decimal places (+0.658, -0.448)

4.10 Run Metadata Block

A compact key-value strip for audit traceability.

Shape: border-radius: 6px
Padding: 10px 14px
Font: mono, 11px

Background: subtle panel color
Layout: Flex-wrap row, gap: 4px 20px
Keys: muted color, followed by colon
Values: weight 600, primary color

4.11 Legend

A horizontal row of color swatches with labels.

Layout: Flex row, gap: 14px
Swatch: 8x8px square, border-radius: 2px
Label: sans, 10px, muted color

4.12 Warning Banner

A colored alert strip for important callouts (row count changes, null injection).

Shape: border-radius: 6px
Padding: 8px 14px
Font: sans, 11px
Icon: ⚠ (bold weight 800)
Background: warn.subtle
Border: 1px solid warn.border
Text color: warn.bg (#F59E0B)

4.13 Threshold Annotation

Inline text showing method + threshold + observed value for audit.

Font: mono, 10px, muted color
Format: "{method} · threshold: {threshold} · value: {value}"
Example: "csi_numeric · threshold: 0.2500 · value: 1.8775"

5. `.show()` — Data Profile View

5.1 Page Layout (top to bottom)

HEADER: mandos logo + "DATA PROFILE" badge
RunMeta: snapshot, table, rows, cols, run

VERDICT BANNER

METRIC CARDS ROW:
[Rows] [Missing] [Duplicates] [Numeric]

\$1. Data Quality Flags
Table: Feature | Metric | Value | Status
| Threshold Annotation

\$2. Column Profiles
For each flagged/interesting feature:

FEATURE_NAME	[numeric]	[status]
Stats	Distribution	
count	bar chart	
missing		
mean		
std		
min/max		
P25/P50		
P75/P99		
skewness		

\$3. Score Correlations
Diverging correlation bars (Spearman ρ)
Sorted by absolute magnitude descending

5.2 Column Profile Card

Each card has:

- **Header bar:** Feature name (mono, bold) + type badge ("numeric"/"categorical") + optional status badge
- **Body:** Two-column flex layout
 - **Left:** Key-value stat pairs (count, missing, mean, std, min, percentiles, max, skewness, kurtosis, zero_count)
 - **Right:** Single-dataset distribution bar chart

5.3 Data

- Rows come from `mandos.profile()` results
 - DQ table rows come from triage results filtered to CRITICAL and WARN
 - Correlations come from Spearman rank correlation with the score column
 - Show the top N features sorted by severity, not all 24
-

6. `.compare()` — Dataset Comparison View

6.1 Page Layout (top to bottom)

HEADER: mandos logo + "DATASET COMPARISON"
RunMeta: comparison name, reference table, current table, filter, run time
VERDICT BANNER
METRIC CARDS ROW: [Drifted] [Schema] [Row Count] [Null Inj]
WARNING BANNERS (if any): ▲ Row count decreased by -92.4% ▲ Possible NULL injection in MEAN_VANT...
\$1. Drift Overview Horizontal CSI bar chart (all features) Legend: Critical / High / Moderate
\$2. Drift Details Table: Feature Type Method CSI Status Ref vs Current chart (inline dual distribution in last column) Legend: Reference (blue) / Current (orange)
\$3. Metric Deltas Table: Feature Metric Reference Current Delta (colored) Shows mean, std, min, max, distinct_count changes for critical features
\$4. Null Rate Changes Table: Feature Ref Null% Cur Null%

Delta Status
\$5. Distribution Deep Dive
2x2 grid of top 4 drifted features
Each cell: name, CSI value, large dual distribution chart, legend

6.2 Key Design Details

- **Drift overview bars** are sorted by CSI descending. Color-coded by severity band.
- **Drift detail table** includes an inline `DistChart` (dual overlay) in the last column. This eliminates the need to scroll to a separate chart section.
- **Type badge** in drift table: purple for categorical, indigo for numeric.
- **Metric deltas** use the `Delta` component: green for small changes, amber for moderate, red for large. Signed format with 4 decimal places or human-readable (e.g., "+9,174", "-28.66").
- **Distribution deep dive** cards have a red-tinted border (`critical.border`) and subtle red background (`critical.glow`).

6.3 Data

- Drift data comes from `mandos.compare_snowflake()` results
- CSI values come from `csi_numeric` or `csi_categorical` metrics
- Metric deltas come from the comparison's per-feature stats (mean, std, min, max, null_rate, distinct_count)
- Null rate changes come from `null_rate_delta` metrics

7. `.report()` — Audit PDF View

7.1 Page Layout (top to bottom)

FORMAL HEADER:
Left: "Mandos Model Monitoring Report"
Right: timestamp, Run ID
Below: RunMeta block (model, version,

baseline, current, columns, tool) Bottom border: 3px solid #4F46E5
\$1. Verdict Verdict banner (light theme)
\$2. Health Board Cards: [Drifted] [DQ Issues] [Schema] [Null Injection] Table: Indicator Kind Status Value
\$3. Issues – Drift CSI bar chart (top 8) Legend Drift issues table
\$4. Issues – Data Quality DQ issues table with Reason column
\$5. Appendix – Run Metadata Full config dump in mono font Model, version, baseline reference, run ID, tool version, thresholds

7.2 Report-Specific Design

- **Header** uses a 3px indigo bottom border as a formal separator.
- **Title** is “Mandos Model Monitoring Report” in gradient indigo, 22px, weight 800.
- **RunMeta** is more verbose than notebook views — includes model name, version, baseline reference description, tool version.
- **Appendix** lists every configuration parameter for reproducibility. This satisfies SR 11-7 documentation requirements.
- **No interactive elements.** Everything is static for PDF rendering.

7.3 SR 11-7 Compliance Features

The report format addresses key SR 11-7 expectations:

1. **Model identification:** Appendix includes model name, version, baseline reference, run ID.
2. **Validation documentation:** Every metric shows method + threshold + value.

3. **Ongoing monitoring:** Drift and DQ sections provide evidence of production monitoring.
 4. **Effective challenge:** Verdict banner forces a clear pass/fail determination.
 5. **Audit trail:** RunMeta captures who ran what, when, against which data.
-

8. Rendering Implementation

8.1 `.show()` and `.compare()` — Notebook HTML Widget

The output is rendered as an inline HTML widget in Jupyter using `IPython.display.HTML`.

CRITICAL: Kernel crash prevention for enterprise JupyterHub:

```
from IPython.display import HTML, display

class MandosDisplay:
    """Renders Mandos output as a self-contained HTML widget."""

    def __init__(self, html_content: str, max_height: int = 800):
        self.html = html_content
        self.max_height = max_height

    def _repr_html_(self):
        """Return an iframe-wrapped version to isolate CSS/JS."""
        # IMPORTANT: Use srcdoc iframe to prevent CSS bleed
        # and reduce notebook cell output size
        import html as html_mod
        escaped = html_mod.escape(self.html)
        return f'''
<iframe
    srcdoc="{escaped}"
    style="width:100%; height:{self.max_height}px; border:none; border-ra
    sandbox="allow-scripts"
></iframe>
'''
```

Known causes of kernel crashes on JupyterHub:

Cause	Symptom	Fix
HTML output too large (>5MB)	Kernel dies on <code>display()</code>	Paginate output; limit to top-N features; lazy-load sections

Inline base64 images	Memory spike	Use SVG for all charts instead of PNG/base64
IOPub rate limit	IOPub data rate exceeded then crash	Set -- ServerApp.iopub_data_rate_limit=1e10 in JupyterHub config, OR reduce output size
Plotly/Bokeh JS payload	Large JS bundle in cell output	Use pure SVG/HTML, no JS libraries
nbconvert conflict	Rendering conflict with notebook extensions	Use iframe isolation (srcdoc)
Memory on shared server	OOM kill from container limits	Profile only flagged features, not all columns

Recommended architecture for .show() and .compare():

```
def show(self):
    """Display profile results in notebook."""
    # 1. Build HTML from template strings (no Jinja – too heavy)
    # 2. All charts are inline SVG (no matplotlib, no plotly, no JS)
    # 3. Wrap in iframe via srcdoc for CSS isolation
    # 4. Cap output: show only flagged features + top-N by importance
    # 5. Total HTML size target: < 500KB

    html = self._build_html() # returns complete HTML document string

    # Guard against oversized output
    size_kb = len(html.encode('utf-8')) / 1024
    if size_kb > 2000:
        # Paginate: show summary + "expand" message
        html = self._build_summary_html()

    display(HTML(self._wrap_iframe(html)))
```

8.2 .report() — PDF Generation

```
def report(self, output_path: str = None) -> Path:
    """Generate audit-grade PDF report."""
    # 1. Build HTML with light theme
    # 2. Convert to PDF using weasyprint or playwright
    # 3. Return file path
```

```

html = self._build_report_html()

# Option A: weasyprint (pure Python, no browser needed)
from weasyprint import HTML as WPHTML
WPHTML(string=html).write_pdf(output_path)

# Option B: playwright (better CSS support, heavier)
# from playwright.sync_api import sync_playwright
# with sync_playwright() as p:
#     browser = p.chromium.launch()
#     page = browser.new_page()
#     page.set_content(html)
#     page.pdf(path=output_path, format='Letter')

```

8.3 Chart Rendering — Pure SVG

All charts **MUST** be rendered as inline SVG strings. No matplotlib, no plotly, no bokeh, no JS charting libraries.

Why: JS libraries bloat the HTML output (plotly.js is ~3MB), cause IOPub rate limit crashes, and don't render in PDF export. SVG is lightweight, print-ready, and renders identically in notebook and PDF.

```

def _svg_bar_chart(
    data: list[float],
    width: int = 200,
    height: int = 50,
    color: str = "rgba(99,162,241,0.6)",
    bar_radius: int = 1,
) -> str:
    """Generate an inline SVG bar chart."""
    n = len(data)
    max_val = max(data) if data else 1
    bar_w = width / n - 1
    bars = []
    for i, v in enumerate(data):
        bar_h = (v / max_val) * height if max_val > 0 else 0
        x = i * (bar_w + 1)
        y = height - bar_h
        bars.append(
            f'<rect x="{x}" y="{y}" width="{bar_w}" '
            f'height="{bar_h}" fill="{color}" rx="{bar_radius}" />'
        )
    return f'<svg width="{width}" height="{height}">{" ".join(bars)}</svg>'

```

```

def _svg_dual_bar_chart(
    ref_data: list[float],
    cur_data: list[float],
    width: int = 200,
    height: int = 50,
    ref_color: str = "#6366F1",
    cur_color: str = "#F97316",
) -> str:
    """Generate a grouped bar chart comparing ref vs current."""
    n = len(ref_data)
    max_val = max(max(ref_data), max(cur_data)) if ref_data else 1
    bin_w = width / n
    bar_w = bin_w * 0.42
    bars = []
    for i in range(n):
        # Reference bar (left)
        rh = (ref_data[i] / max_val) * height
        bars.append(
            f'<rect x="{i * bin_w + 1}" y="{height - rh}" '
            f'width="{bar_w}" height="{rh}" '
            f'fill="{ref_color}" opacity="0.6" rx="1"/>'
        )
        # Current bar (right)
        ch = (cur_data[i] / max_val) * height
        bars.append(
            f'<rect x="{i * bin_w + bar_w + 2}" y="{height - ch}" '
            f'width="{bar_w}" height="{ch}" '
            f'fill="{cur_color}" opacity="0.7" rx="1"/>'
        )
    return f'<svg width="{width}" height="{height}">{"".join(bars)}</svg>'

```

8.4 HTML Template Structure

All three views use a single `_build_html()` pipeline:

```

def _build_html(self, theme: str = "dark") -> str:
    """Build complete HTML document.

    Args:
        theme: "dark" for .show()/.compare(), "light" for .report()
    """
    css = self._css(theme)
    body = self._body(theme)

    return f'<!DOCTYPE html>

```



```

<html>
<head>
<meta charset="utf-8">
<style>{css}</style>
<link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;6
</head>
<body>{body}</body>
</html>""

```

Offline font fallback: On enterprise JupyterHub behind firewalls, Google Fonts may be blocked. Include a fallback:

```

/* Fallback if Google Fonts is unreachable */
@font-face {
    font-family: 'IBM Plex Sans';
    src: local('IBM Plex Sans'), local('IBMPlexSans');
}
@font-face {
    font-family: 'JetBrains Mono';
    src: local('JetBrains Mono'), local('JetBrainsMono');
}

```

If neither loads, the CSS fallback chain (`-apple-system, sans-serif / monospace`) handles it gracefully.

9. Plot Type Reference

Plot	Used In	Type	Implementation
Single distribution	<code>.show()</code> column profiles	Vertical bars	<code>_svg_bar_chart()</code>
Dual distribution overlay	<code>.compare()</code> drift table, deep dive	Grouped vertical bars	<code>_svg_dual_bar_chart()</code>
CSI horizontal bars	<code>.compare()</code> , <code>.report()</code>	Horizontal bars	HTML div with <code>width%</code> fill
Diverging correlation	<code>.show()</code> correlations	Diverging horizontal bar	HTML div, centered at 50%
	<code>.show()</code> ,	Stacked	

DQ rate bars	<code>.report()</code> DQ section	horizontal bars	HTML divs (capping, flooring, missing, zero rates)
--------------	-----------------------------------	-----------------	--

All plots are inline SVG or CSS-only. No canvas, no JS, no external libraries.

10. Responsive Behavior

10.1 Notebook Width

Jupyter cells are typically 700-1000px wide. Design for 900px max content width.

- Metric cards: `flex: 1 1 130px` with `flex-wrap: wrap` — wraps to 2 rows on narrow cells
- Distribution deep dive: `grid-template-columns: 1fr 1fr` — stacks to `1fr` below 600px
- Tables: horizontal scroll via `overflow-x: auto` on container
- CSI bars: feature name width capped at 200px with `text-overflow: ellipsis`

10.2 PDF Width

US Letter with 1" margins = ~6.5" content width (~624px at 96dpi). The light theme is designed for this width.

11. Mandos Logo Treatment

```
Text:      "mandos"
Font:      IBM Plex Sans, 18px, weight 800
Style:     background: linear-gradient(135deg, #6366F1, #A78BFA)
           -webkit-background-clip: text
           -webkit-text-fill-color: transparent
Spacing:   letter-spacing: -0.03em
```

Adjacent to the logo, a method badge:

```
Text:      "DATA PROFILE" | "DATASET COMPARISON" | "MODEL MONITORING REPORT"
Style:     10px, padding 2px 8px, border-radius 4px
           background: rgba(99,102,241, 0.12)
           color: #A78BFA
```

```
font-weight: 600
letter-spacing: 0.04em
```

In `.report()` (light theme), the title is larger (22px) and rendered as a full line “Mandos Model Monitoring Report” with a gradient color fill from `#4F46E5` to `#6366F1`.

12. Implementation Checklist

For `.show()` :

- Header with logo + DATA PROFILE badge + RunMeta
- Verdict banner with triage summary
- 4 metric cards (rows, missing, duplicates, numeric count)
- §1 DQ flags table with threshold annotations
- §2 Column profile cards (stats + distribution chart)
- §3 Correlation diverging bars
- Total HTML < 500KB
- All charts are inline SVG
- Output wrapped in iframe via `_repr_html_`

For `.compare()` :

- Header with logo + DATASET COMPARISON badge + RunMeta
- Verdict banner
- 4 metric cards (drifted, schema, row count, null injection)
- Warning banners for row count changes and null injection
- §1 CSI bar chart with legend
- §2 Drift detail table with inline dual distribution charts
- §3 Metric deltas table
- §4 Null rate changes table
- §5 Distribution deep dive (2×2 grid, top 4)

- Total HTML < 500KB
- All charts are inline SVG

For `.report()` :

- Formal header with title, timestamp, Run ID
- Verbose RunMeta block
- §1 Verdict
- §2 Health Board (cards + checkpoint table)
- §3 Drift (CSI bars + drift issues table)
- §4 Data Quality (DQ issues table with reasons)
- §5 Appendix (full config dump)
- PDF export via weasyprint or playwright
- Light theme throughout
- SR 11-7 traceability: every metric has method + threshold + value